

操作系统强化课考试

一、某男子足球俱乐部，有非常多的教练、队员。每次足球训练开始之前，教练、球员都需要先进入更衣室换衣服，可惜俱乐部只有一个更衣室。教练们脸皮薄，无法接受和别人共用更衣室。队员们脸皮厚，可以和其他队员一起使用更衣室。更衣完成后，才可以参加足球训练。

1) 起初，俱乐部规定：如果队员和教练都要使用更衣室，则应该让教练优先。请使用P、V操作描述上述过程的互斥与同步，并说明所用信号量及初值的含义。

2) 后来，因很多队员无法忍受教练优先的特权现象，俱乐部更改了规定——为体现“人人平等”的团队作风，队员和教练应遵循“先到先得”的原则，公平地使用更衣室。请使用P、V操作描述上述过程的互斥与同步，并说明所用信号量及初值的含义。

【参考答案】本题中，教练就是写者，队员就是读者，不过是读者写者问题换了个马甲而已。

1) 按照题目要求，要求实现“写优先”。“写优先”可能导致读者饥饿，参考代码如下：

```
1  semaphore readLock=1;           //互斥信号量，用于给读者“上锁”
2  semaphore writeLock=1;          //互斥信号量，用于给写者“上锁”
3  semaphore rmutex=1;             //互斥信号量，实现对readCount的互斥访问
4  semaphore wmutex=1;             //互斥信号量，实现对writeCount的互斥访问
5  int readCount=0, writeCount=0;  //读者、写者的数量
6
7  //读者进程（在这个题里就是可以多人一起共用更衣室的队员们）
8  Reader(){
9  while(true){
10     P(readLock);                 //每个读者到达时先对 read 上锁
11     P(rmutex);
12     readCount++;
13     if(readCount==1) P(writeLock); //第一个开始读的读者对写者上锁
14     V(rmutex);
15     V(readLock);                 //每个读者正式开始读之前对 read 解锁
16     读者读...;
17     P(rmutex);
18     readCount--;
19     if(readCount==0) V(writeLock); //最后一个读完的读者对写者解锁
20     V(rmutex);
21 }
22 }
23
24 //写者进程（在这个题目里，对应必须独享更衣室的教练们）
25 Writer(){
26 while(true){
27     P(wmutex);
28     writeCount++;
29     if(writeCount==1) P(readLock); //第一个到达的写者对读者上锁，这一步是实现“写优先”的关键
30     V(wmutex);
```

```

31 P(writeLock); //每个写者开始写之前都要 P(write), 保证写者之间互斥, 同时也能保证若当前
    有读者正在读, 那么写者等待
32 写者写...;
33 V(writeLock);
34 P(wmutex);
35     writeCount--;
36     if(writeCount==0) V(readLock); //最后一个写者写完之后, 对读者解锁
37 V(wmutex);
38 }
39 }

```

为了方便大家对比学习, 下面再附上“读者优先”的实现, 这种方式可能导致写者饥饿:

```

1 semaphore lock=1; //用于实现对共享文件的互斥访问
2 int count = 0; //记录当前有几个读进程在访问文件
3 semaphore mutex = 1; //用于保证对count变量的互斥访问
4
5 writer (){
6 while(1){
7 P(lock); //写之前“加锁”
8 写文件...
9 V(lock); //写完了“解锁”
10 }
11 }
12
13 reader (){
14 while(1){
15 P(mutex); //各读进程互斥访问count
16     if(count==0)
17         P(lock); //第一个读者, 读之前“上锁”
18     count++; //访问文件的读进程数+1
19 V(mutex);
20 读文件...
21 P(mutex); //各读进程互斥访问count
22     count--; //访问文件的读进程数-1
23     if(count==0) //由最后一个读进程负责
24         V(lock); //读完了“解锁”
25 V(mutex);
26 }
27 }

```

2) 这个小问要求实现“读写公平”。参考代码如下:

```

1 semaphore lock=1; //用于实现对共享文件的互斥访问
2 int count = 0; //记录当前有几个读进程在访问文件
3 semaphore mutex = 1; //用于保证对count变量的互斥访问
4 semaphore queue = 1; //用于实现“读写公平”。咸鱼注: 可以将queue理解为一个“队列”, 当资源
    暂不可访问时, 无论读者、写者都需要公平排队
5
6 writer (){
7 while(1){

```

```

8   P(queue);    //先排队
9   P(lock);     //尝试“上锁”
10  V(queue);    //唤醒下一个队头进程
11  写文件...
12  V(lock);     //使用完资源，解锁
13  }
14  }
15
16  reader () {
17  while(1) {
18      P(queue);    //先排队
19      P(mutex);    //互斥访问count
20      if(count==0)
21          P(lock);    //第一个到达的读者尝试“上锁”
22      count++;      //读者计数+1
23      V(mutex);
24      V(queue);    //唤醒队头进程
25      读文件...
26      P(mutex);    //互斥访问count
27      count--;      //读者计数-1
28      if(count==0)    //最后一个离开的读者，负责“解锁”
29          V(lock);
30      V(mutex);
31  }
32  }

```

对于上面三种解法，如果弄不清楚它们之间的区别，不妨带入一个例子看看。假设读者的读操作耗时较长，读者写者到达的顺序是：

读者1——读者2——读者3——写者A——读者4——写者B——读者5

如果采用“写优先”的实现方法，那情况是这样的：读者1到达并开始读，紧接着读者2、读者3到达，都可以开始读；写者A到达，暂时不能写；读者4到达，暂时不能读；写者B到达，暂时不能写；读者5到达，暂时不能读；等读者1、2、3都读完之后，写者A开始写；写者A写完之后写者B开始写；写者B写完后读者4开始读，同时读者5也可以开始读。

如果采用“读者优先”的实现方法，那情况是这样的：读者1到达并开始读，紧接着读者2、读者3到达，都可以开始读；写者A到达，暂时不能写；读者4到达，可以开始读；写者B到达，暂时不能写；读者5到达，可以直接开始读；等读者1、2、3、4、5都读完之后，写者A、写者B才可以依次进行写。

如果采用“读写公平法”的实现方法，那情况是这样的：读者1到达并开始读，紧接着读者2、读者3到达，都可以开始读；写者A到达，暂时不能写；读者4到达，暂时不能读；写者B到达，暂时不能写；读者5到达，暂时不能读；等读者1、2、3都读完之后，写者A开始写；写者A写完之后读者4开始读；读者4读完后写者B开始写；写者B写完后读者5开始读。

二、俗话说，“干饭人，干饭魂，干饭人吃饭得用盆”。一荤、一素、一汤、一米饭，是每个干饭人的标配。饭点到了，很多干饭人奔向食堂。每个干饭人进入食堂后，需要做这些事：拿一个盆打荤菜，再拿一个盆打素菜，再拿一个盆打汤，再拿一个盆打饭，然后找一个座位坐下干饭，干完饭把盆还给食堂，然后跑路。现在，食堂里共有N个盆，M个座位。请使用P、V操作描述上述过程的互斥与同步，并说明所用信号量及初值的含义。

【参考答案】

显然，这个题目的关键是不能发生死锁。死锁问题应该参考哲学家问题的解决思路。在哲学家进餐问题中，我们解决死锁的方法有三种：

1. 奇数号哲学家必须先拿左手的筷子，偶数号哲学家必须先拿右手的筷子
2. 限制“最多允许4个哲学家同时进餐”
3. 仅当一个哲学家左右两边的筷子都可用时才允许哲学家拿筷子

其中，第一种方法的思想很难迁移到其他题目中。但是第二、第三种思想可以迁移到大多数死锁题目。

我们先来看一个标准的错误解法：

```
1 semaphore pot=N; //同步信号量，用于表示“盆”资源，食堂里总共有N个盆
2 semaphore seat=M; //同步信号量，用于表示“作为”资源，食堂里总共有M个座位
3
4 //干饭人进程
5 EatMan(){
6     进食堂;
7     P(pot); //拿一个盆
8     打荤菜;
9     P(pot); //拿一个盆
10    打素菜;
11    P(pot); //拿一个盆
12    打汤;
13    P(pot); //拿一个盆
14    吃饭;
15    P(seat); //占个座
16    干饭;
17    V(seat); //让出座位
18    V(pot); //归还干饭盆
19    V(pot);
20    V(pot);
21    V(pot);
22    离开食堂;
23 }
```

显然，上面这种解法会导致死锁。假设同时来了好多个干饭人，每个人都拿三个盆，盆很快就会被拿光。那所有人都无法得到第四个盆，就会发生死锁。

下面我们模仿哲学家进餐问题的第二种解决思路。在哲学家问题中，共有5个哲学家，如果我们限制“最多允许4个哲学家同时进餐”，那么至少会有一个哲学家可以同时获得左右两只筷子，并顺利进餐，从而预防了死锁。

同样的思路可以迁移到干饭人问题中。每个干饭人需要同时持有4个盆才能干饭，那么最糟糕的情况是每个干饭人都持有3个盆，同时在等待第四个盆。此时，但凡再多一个盆，就至少能有一个干饭人可以顺利干饭，就不会死锁。因此我们可以限制同时抢盆的人数为 x ，那么只要满足 $3x + 1 \leq N$ ，则一定不会发生死锁，可得 $x \leq (N-1)/3$ 。参考代码如下：

```

1 semaphore pot=N;           //同步信号量，用于表示“盆”资源，食堂里总共有N个盆
2 semaphore seat=M;         //同步信号量，用于表示“作为”资源，食堂里总共有M个座位
3 semaphore x=(N-1)/3;      //同步信号量x，用于表示最多允许多少个人同时干饭。(N-1)/3 向下取整
4 //干饭人进程
5 EatMan() {
6     P(x);                 //进食堂拿盆之前，先看看是否已到达人数上限
7     进食堂;
8     P(pot);               //拿一个盆
9     打荤菜;
10    P(pot);               //拿一个盆
11    打素材;
12    P(pot);               //拿一个盆
13    打汤;
14    P(pot);               //拿一个盆
15    干饭;
16    P(seat);              //占个座
17    干饭;
18    V(seat);              //让出座位
19    V(pot);               //归还干饭盆
20    V(pot);
21    V(pot);
22    V(pot);
23    V(x);
24    离开食堂;
25 }

```

上面这种做法，限制了人数上限，且先拿盆，再占座，一定不会发生死锁。当然，如果先占座、后拿盆，也不会死锁。事实上，如果座位的数量满足 $seat \leq (N-1)/3$ ，那么甚至可以不设置专门的信号量x，完全可以先占座，后拿盆，也一定不会死锁。因为座位的数量就可以限制同时抢盆的人数。

下面我们再模仿哲学家问题的第三种解决思路——仅当一个哲学家左右两边的筷子都可用时才允许哲学家拿筷子。破坏了“请求和保持”条件，采用“静态分配”的思想，让进程一口气获得所有资源，再开始运行。代码如下：

```

1 semaphore mutex=1;        //互斥信号量，保证所有进程对 pot 变量、seat 变量的访问是互斥的。
2 semaphore pot=N;          //用于表示“盆”资源，食堂里总共有N个盆
3 semaphore seat=M;         //用于表示“作为”资源，食堂里总共有M个座位
4
5 //干饭人进程
6 EatMan() {
7     进食堂;
8     P(mutex);
9     P(pot);               //一口气拿四个盆，并占座
10    P(pot);
11    P(pot);
12    P(pot);
13    P(seat);
14    V(mutex);
15
16    打荤菜;
17    打素材;

```

```
18    打汤;
19    打饭;
20    干饭;
21
22    V(seat);    //让出座位
23    V(pot);     //归还干饭盆
24    V(pot);
25    V(pot);
26    V(pot);
27    离开食堂;
28 }
```

这个题目想告诉大家的是，哲学家进餐问题的解决思路中，后两种方法更为通用，可以作为考试时主要的策略。大家再思考一下，限制人数上限、一口气拿所有资源，哪种方案的并发度更高一些呢？显然是后者对吧。“限制人数上限”的方案中，最糟糕的情况是，只有一个人获得了4个盆，其余进程都只有3个盆，也就是说只有1个进程可以顺利运行下去，因此并发度低。

最后，再使用强化课P2给大家的模板，也就是用int变量表示资源

哲学家问题(特点:进程,要占用多种资源才能运行)

模板1:用int变量表示资源(并发度高)

模板2:用信号量表示资源(并发度不高) X

① 定义大锁 $\Rightarrow$ Semaphore Lock = 1; //互斥信号量

② 定义资源数 int $\Rightarrow$ 如:有a,b,c三类资源,分别有9个,8个,5个,则
定义3个int变量.
int a=9; //表示a的剩余数量.
int b=8; //表示b的剩余数量.
int c=5; //表示c的剩余数量.

写代码模板:

Process () {

③ 一口气拿所有资源 { while(1) {
P(Lock);
if(所有资源都够) {
所有资源int值减少; //题目会告诉你每类资源要几个.
取xxx资源; //一口气拿走所有资源
V(Lock); //拿完资源,解锁
break; //跳出while循环
}
V(Lock); //资源不够,解锁,再循环尝试一次.
} //while结束

④ $\rightarrow$ 做进程该做的事(如:哲学家干饭); //用中文说明即可

⑤ 一口气归还所有资源 { P(Lock);
归还所有资源,所有资源int值增加;
V(Lock);

} // End

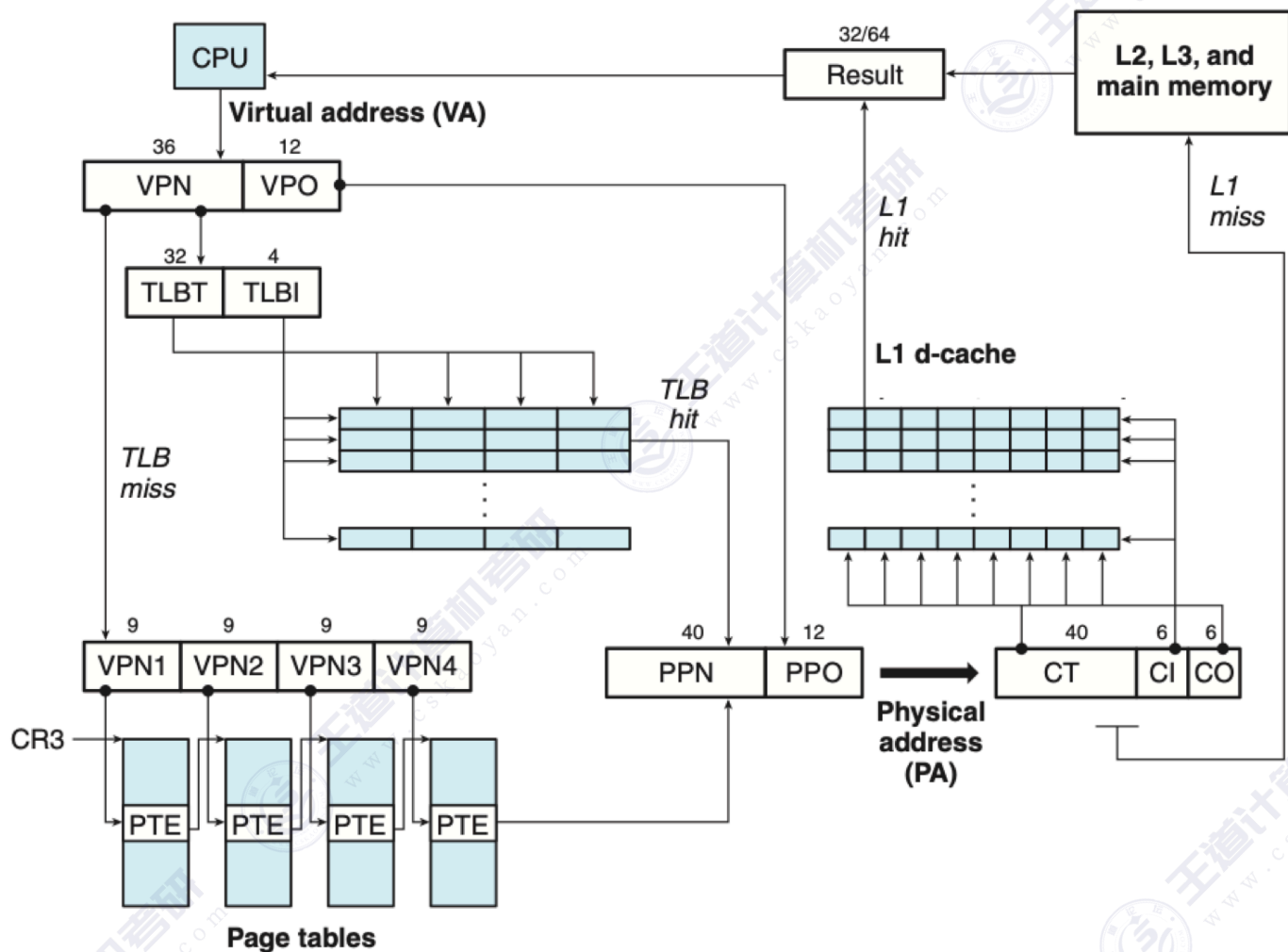
```
1 semaphore mutex=1; //互斥信号量,保证所有进程对 pot 变量、seat 变量的访问是互斥的。
2 int pot=N; //用于表示“盆”资源,食堂里总共有N个盆
3 int seat=M; //用于表示“作为”资源,食堂里总共有M个座位
4
5 //干饭人进程
6 EatMan() {
7     进食堂;
8
9     while(1) {
```

```

10     P(mutex);    //资源上锁
11     if ( pot>=4 && seat>=1 ) {    //所有资源都够
12         pot -=4;    //一口气拿走所有资源
13         seat -=1;
14         V(mutex); //拿完资源，解锁
15         break;    //拿完资源，跳出循环
16     }
17     V(mutex);    //资源不够，解锁
18 } //while
19
20 打荤菜;
21 打素材;
22 打汤;
23 打饭;
24 干饭;
25
26     P(mutex);    //资源上锁
27     pot +=4;    //一口气归还所有资源
28     seat +=1;
29     V(mutex);    //资源解锁
30
31 离开食堂;
32 }

```

三、下面是 Intel Core i7 地址转换过程的简要示意图，已知该机按字节编址，请根据图示回答下面的问题：



- 1) 该CPU支持的虚拟地址空间大小是多少？物理地址空间大小是多少？
- 2) 该处理器支持几级页表？每个页面的大小是多少？每个页表项的大小是多少？
- 3) 若TLB命中率为98%，TLB访问时间1ns，内存访问时间100ns。并假设当TLB miss时才开始访问内存中的页表，且各级页表常驻内存，则平均的地址转换时间是多少？
- 4) TLB访问时间1ns，内存访问时间100ns。并假设当TLB miss 时才开始访问内存中的页表，只有顶级页表常驻内存。查多级页表的过程中，下一级页表未调入内存的概率是 1%。已知处理一个缺页中断平均需要8ms。如果要满足平均地址转换时间小于2ns，那么TLB命中率需要至少多少？
- 5) 假设访问某虚拟地址时，TLB未命中，已查到第四级页表，且第四级页表已经调入内存。访问目标物理地址的过程中，不考虑 Cache，若内存访问时间100ns。当发生缺页时，若有一个可用的空页或被置换的页未被修改，则处理缺页并访问目标物理地址需要5ms；若被置换的页已被修改，则处理缺页并访问目标物理地址的时间需要10ms。请问为什么若被置换页面已被修改，处理缺页的时间开销会增加？若发生缺页时，60%的概率需要置换一个被修改的页面，为保证访问目标物理地址的平均时间不超过1ms，可接受的最大缺页中断率是多少？

【参考答案】

1) 看图可知，虚拟地址VA总位数为 $36+12=48\text{bit}$ ，物理地址PA总位数为 $40+12=52\text{bit}$ 。因此，虚拟地址空间大小为 $2^{48}\text{B}=256\text{TB}$ ，物理地址空间大小为 $2^{52}\text{B}=4\text{PT}$ 。

注：可顺道复习一下计组大纲要求掌握的数据单位 K、M、G、T、P、E、Z。依次递增，每增长一级就是乘 2^{10} 的关系。

2) 看图可知, 该处理器支持4级页表。每个页面大小是 $2^{12}=4\text{KB}$ 。通常来说采用多级页表时, 最理想的情况是每一级的页表大小不超过一个页面, 而从图中可知, 每一级的页号占9位, 也就是说每一级页表有 2^9 个表项, 因此每个页表项的大小为 $2^{12}/2^9=8\text{B}$ 。

3) 若TLB命中, 则可以直接完成地址转换, 耗时1ns。若TLB未命中, 需要查询四级页表, 每查一级页表需要一次访存, 因此地址转换耗时 $=1+100+100+100+100=401\text{ns}$ 。综上, 平均地址转换时间 $=1\times 98\% + 401\times 2\% = 9\text{ns}$

4) 这个小问有点难, 不用深究。通过这个小问, 主要想提醒大家除了一级页表外, 各级页表都有可能缺页:

- 若TLB命中, 则可以直接完成地址转换, 记耗时 $A=1\text{ns}$ 。设TLB命中的概率是 P 。
- 若TLB未命中 (概率 $1-P$) , 需要查询四级页表, 一级页表常驻内存。
 - 概率 $P_1 = (1-P) \times 0.01$ ——如果二级页表未调入内存, 则三、四级页表也一定未调入内存, 查页表过程中累计发生三次缺页。则耗时 $B = (1\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$
 - 概率 $P_2 = (1-P) \times 0.99 \times 0.01$ ——如果二级页表原本已调入内存, 第三级页表未调入内存, 则四级页表也一定未调入内存, 查页表过程中累计发生两次缺页。则耗时 $C = (1\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$
 - 概率 $P_3 = (1-P) \times 0.99 \times 0.99 \times 0.01$ —— 如果三级页表原本已调入内存, 则四级页表未调入内存的概率是1%。查页表过程中累计发生一次缺页。则耗时 $D = (1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$
 - 概率 $P_4 = (1-P) \times 0.99 \times 0.99 \times 0.99$ —— 如果四级页表原本已调入内存, 则查页表过程中不会发生缺页, 则耗时 $E=(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$

平均地址转换时间 $= P \times A + P_1 \times B + P_2 \times C + P_3 \times D + P_4 \times E < 2\text{ns}$, 解出方程式即可得概率TLB命中率 P

5) 若被置换的页面被修改过, 需要现将页面写回外存, 再将缺失页面读入内存。“脏”页面写回外存的动作会增加开销。

查询到第四级页表, 即可确定目标地址是否缺页。考虑两种情况:

- 概率 $= (1-P)$ ——不缺页。访问物理地址耗时 $= 100\text{ns}$ 。
- 概率 $= P$ —— 缺页。此时需要细分为两种情况考虑
 - 概率 $= P \times 40\%$ —— 有空闲页框, 或被置换的页面“不脏”。处理缺页并访问目标物理地址耗时 5ms
 - 概率 $= P \times 60\%$ —— 无空闲页框, 且 被置换的页面“脏”。处理缺页并访问目标物理地址耗时 10ms

综上, 从已查询到四级页表开始算起, 访问目标物理地址的平均耗时 $= (1-P)\times 100\text{ns} + P\times 0.4\times 5\text{ms} + P\times 0.6\times 10\text{ms} < 1\text{ms}$ 。解方程即可得答案

四、下面这个图是 Intel core i7 页表项的结构，每个页表项 大小为8B。其中：

- P：有效位，或者翻译为存在位（Present），表示页面是否已经调入内存，p=0则缺页，p=1则不缺页。
- R/W：用 0/1 表示这个页面是否“只能读”，还是“可读可写”。比如存储常量的页面，就可以设置为只能读。
- U/S：用 0/1 表示这个页面在用户态下是否可访问，还是只有内核态下能访问。
- A：访问位，表示最近有没有被访问过，可以被置换算法使用。
- D：脏位，表示页面信息是否被修改过。当这个页面的数据被“写”过，就脏了。
- 第 12~51 位，共40bit，表示下一级页框号。

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	Unused	Page physical base addr				Unused	G	0	D	A	CD	WT	U/S	R/W	P=1

回答下列问题：

- 1) 结合上一题的条件，判断该CPU可支持的最大物理内存是多少？
- 2) 当用户进程尝试访问操作系统内核区时，系统是如何处理这种非法访问行为的？
- 3) 结合页表项示意图，简述操作系统内核是如何实现改进型 Clock 算法的？
- 4) 第7个bit为0表示这一级页表大小为 4KB；为1表示这一级的页表大小为4MB。但是，只有顶级页表才允许第7个bit为1，为什么？

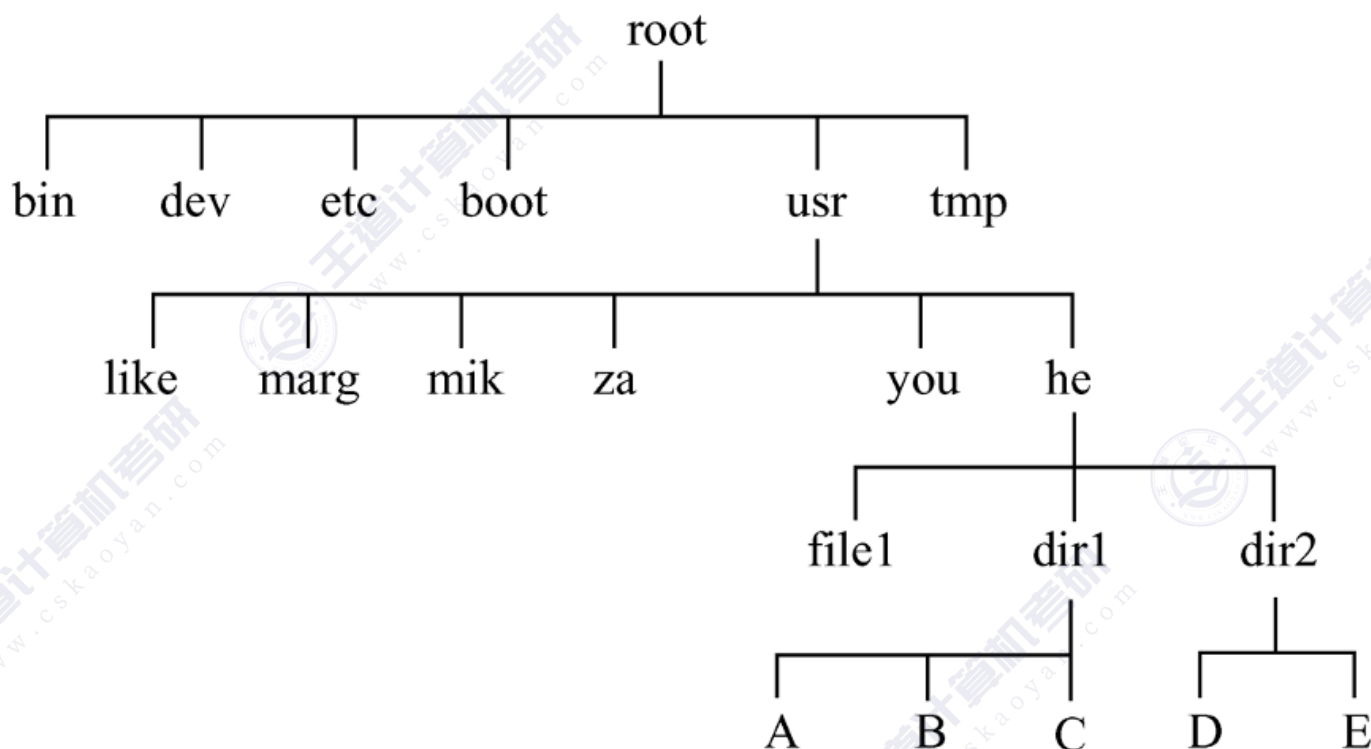
【参考答案】

1. 2^{52} B
2. MMU地址转换机构可通过 U/S 位为 0/1 表示这个页面是否允许在用户态下访问。系统在地址转换的过程中，通过当前CPU的PSW状态位与 U/S 位来判断进程对内核区的访问是否合法。如果不合法，会触发异常。
3. 改进型Clock算法需要考虑进程驻留集中每个页表项的 A, D 两个位。算法规则：将所有可能被置换的页面排成一个循环队列（本质是把对应的页表项排成一个循环队列）。
第一轮扫描：从当前位置开始扫描到第一个 (0, 0) 的页帧用于替换。本轮扫描不修改任何标志位。
第二轮扫描：若第一轮扫描失败，则重新扫描，查找第一个 (0, 1) 的帧用于替换。本轮将所有扫描过的页帧访问位设为0。
第三轮扫描：若第二轮扫描失败，则重新扫描，查找第一个 (0, 0) 的页帧用于替换。本轮扫描不修改任何标志位
第四轮扫描：若第三轮扫描失败，则重新扫描，查找第一个 (0, 1) 的帧用于替换。
由于第二轮已将所有帧的访问位设为0，因此经过第三轮、第四轮扫描一定会有一个帧被选中，因此改进型CLOCK置换算法选择一个淘汰页面最多会进行四轮扫描。
4. 在多级页表的系统中，顶级页表常驻内存，也只有顶级页表的大小可以超过一个页面。这种规定可以让多级页表的管理更简单方便。

五、某个文件系统中，外存为硬盘。物理块大小为512B，有文件A包含598个记录，每个记录占255B，每个物理块放2个记录。文件A所在的目录如下图所示。

文件目录采用多级树形目录结构，由根目录结点、作为目录文件的中间结点和作为信息文件的树叶组成，每个目录项占127B，每个物理块放4个目录项，根目录的第一块常驻内存。试问：

- 1) 若文件的物理结构采用链式存储方式，链指针地址占2B，那么要将文件A读入内存，至少需要存取几次硬盘？
- 2) 若文件为连续文件，那么要读文件A的第487个记录至少要存取几次硬盘？
- 3) 一般为减少读磁盘次数，可采取什么措施？



【参考答案】1) 2) 小问参考答案见王道书 4.2 课后大题5。建议大家认真研究此题，可以帮助大家深入理解连续存储、隐式链接存储。

3) 这个小问题作为思维扩展，不必深究。

可以设置进程的“当前工作目录”，以减少open系统调用时读磁盘的次数。例如：进程P1想要打开文件A，如果从根目录出发，那么查找文件A的FCB，需要查询多级目录的信息，也会引发多次读磁盘操作；而如果设置dir1为进程P1的“当前工作目录”，就意味着dir1的信息已经被提前读入内存，于是，从当前目录出发，找到文件A的FCB就无需读磁盘。

如果要经常对文件的记录进行随机读，相比于第一小问的“隐式链接”方案，更应该采用第二小问“连续存储”的方案，可减少读磁盘次数。对一个文件的随机读，速度上可以这么排序：连续存储>FAT显式链接>混合索引分配>隐式链接

六、有一个文件系统如下图A所示。图中的方框表示目录，圆圈表示普通文件。根目录常驻内存，目录文件组织成链接文件，不设FCB，普通文件组织成索引文件。目录表指示下一级文件名及其磁盘地址（各占2B，共4B）。若下级文件是目录文件，指示其第一个磁盘块地址。若下级文件是普通文件，指示其FCB的磁盘地址。每个目录的文件磁盘块的最后4B供拉链使用。下级文件在上级目录文件中的次序在图中为从左至右。每个磁盘块有512B，与普通文件的一页等长。

普通文件的FCB组织如下图2所示。其中，每个磁盘地址占2B，前10个地址直接指示该文件前10页的地址。第11个地址指示一级索引表地址，一级索引表中每个磁盘地址指示一个文件页地址；第12个地址指示二级索引表地址，二级索引表中每个地址指示一个一级索引表地址；第13个地址指示三级索引表地址，三级索引表中每个地址指示一个二级索引表地址。请问：

- 1) 一个普通文件最多可有多少个文件页？
- 2) 若要读文件J中的某一页，最多启动磁盘多少次？
- 3) 若要读文件W中的某一页，最少启动磁盘多少次？
- 4) 根据3)，为最大限度减少启动磁盘的次数，可采用什么方法？此时，磁盘最多启动多少次？

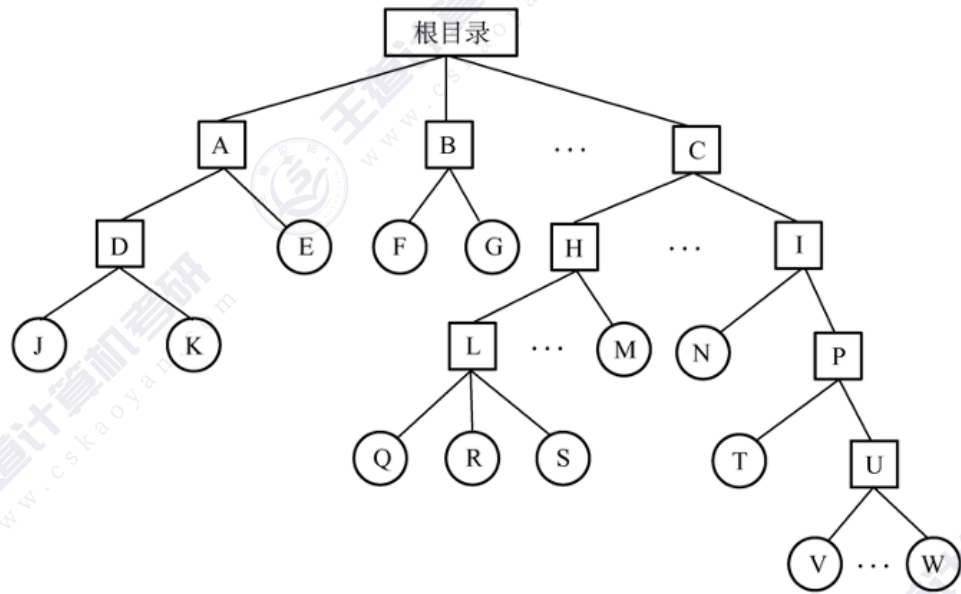


图 A 某树形结构文件系统框图

该文件的有关描述信息	
1	磁盘地址
2	磁盘地址
3	磁盘地址
⋮	⋮
11	磁盘地址
12	磁盘地址
13	磁盘地址

图 B FCB 组织

【参考答案】见王道书 4.2 课后大题 4。